

certmitm

automatic exploitation of TLS
certificate validation
vulnerabilities

aapo (oksman)

Nixu Cybersecurity

- Senior Security Specialist
- Devices (IoT)
- Cryptography
- Network protocols

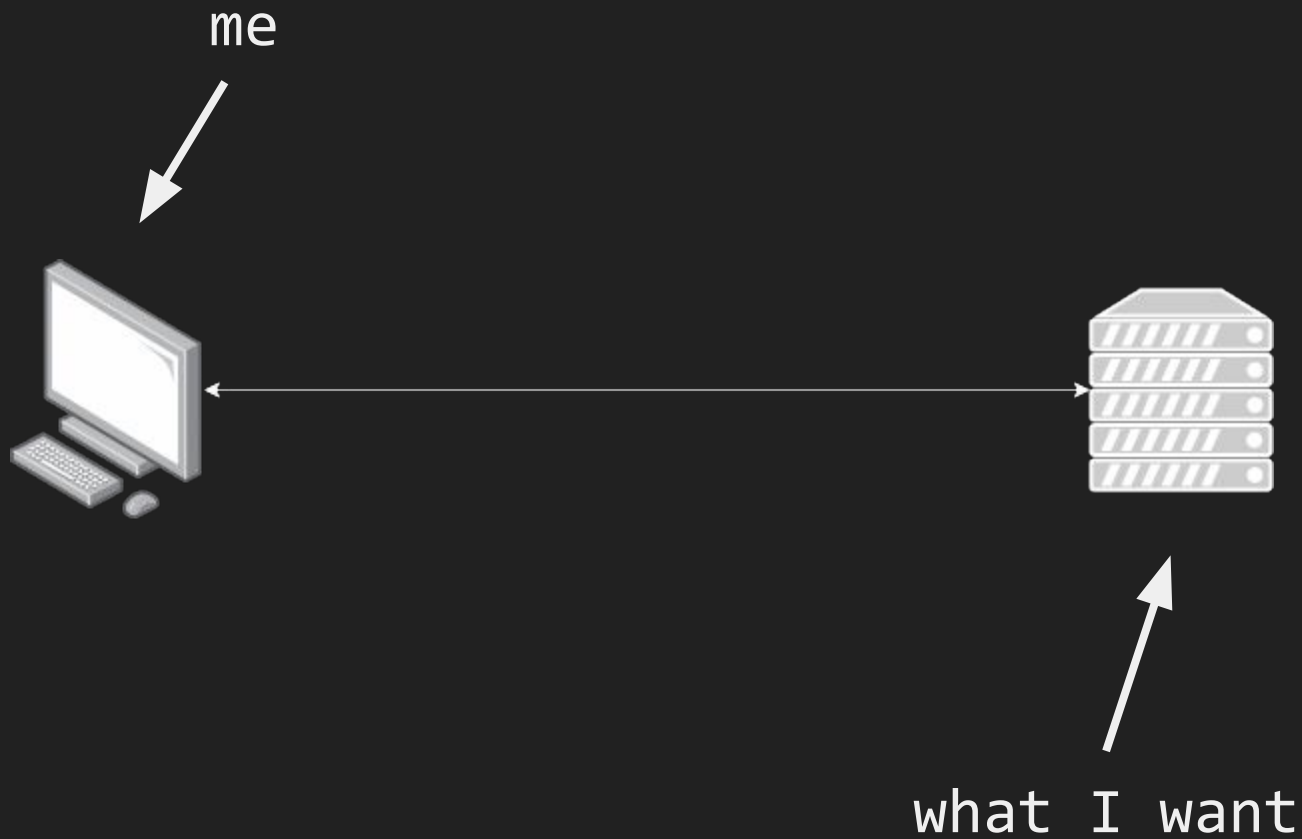
Security Research & Bug Bounty

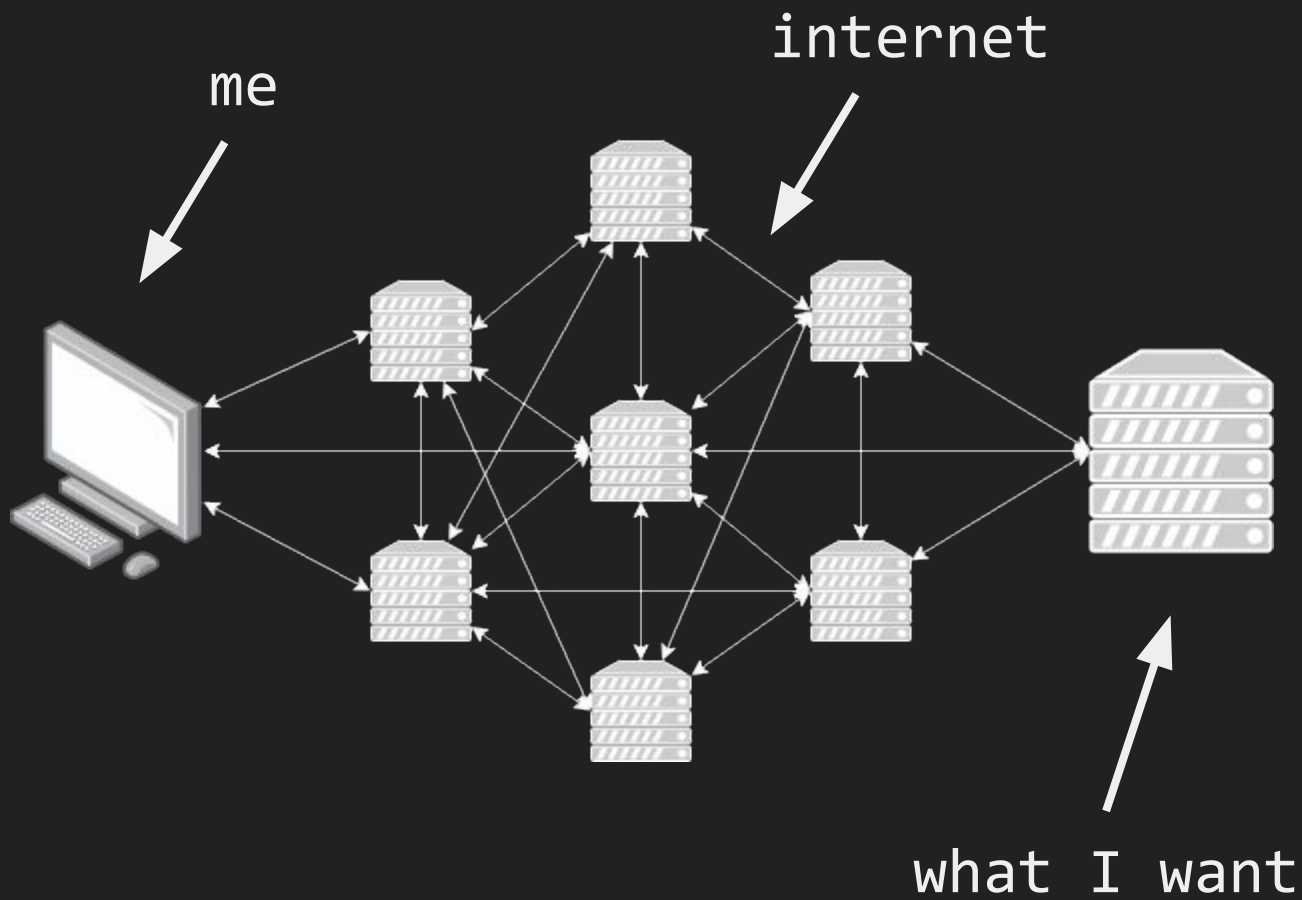
- aapo @ HackerOne/Bugcrowd/Intigriti

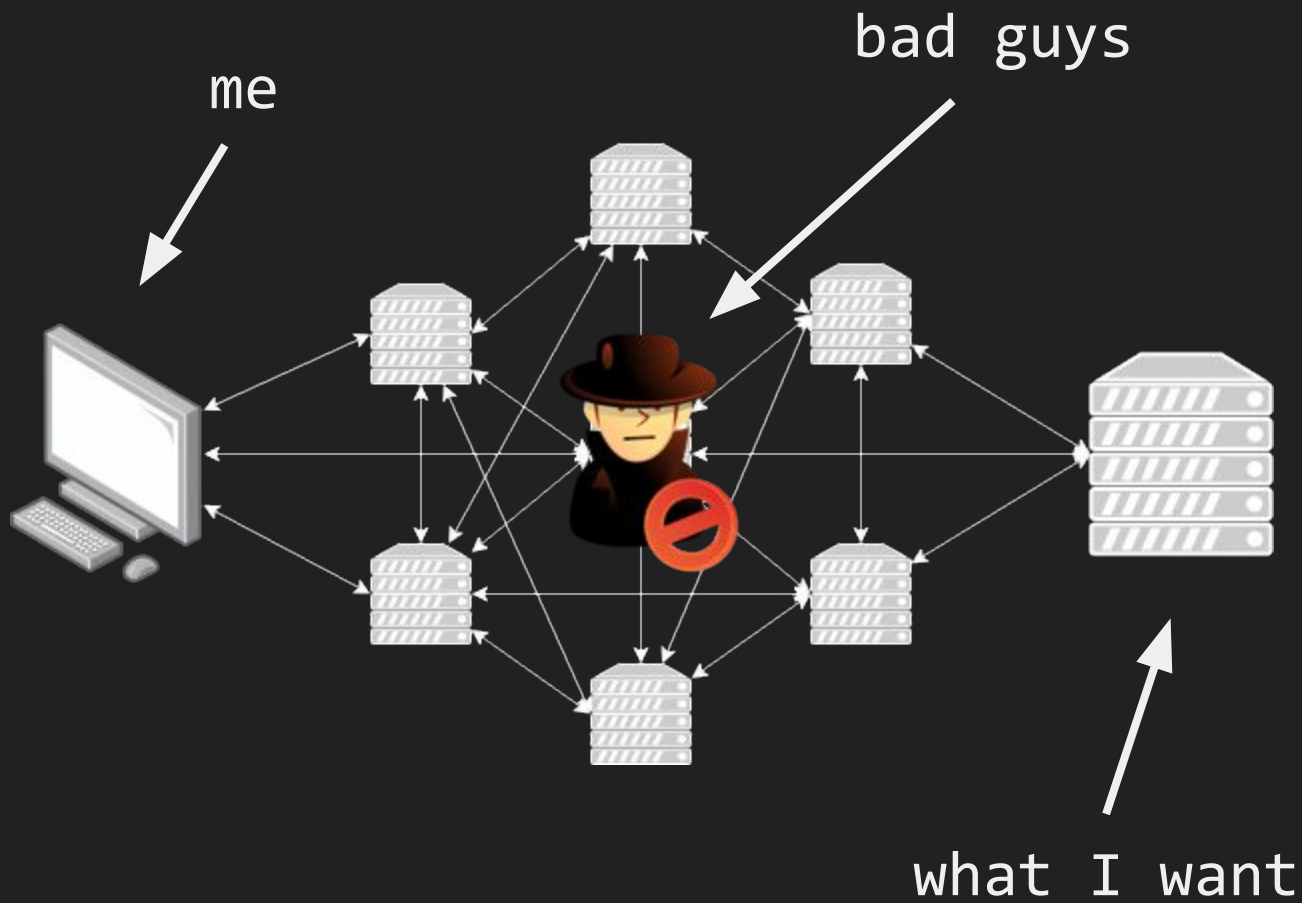
ECSC Team Finland Coach

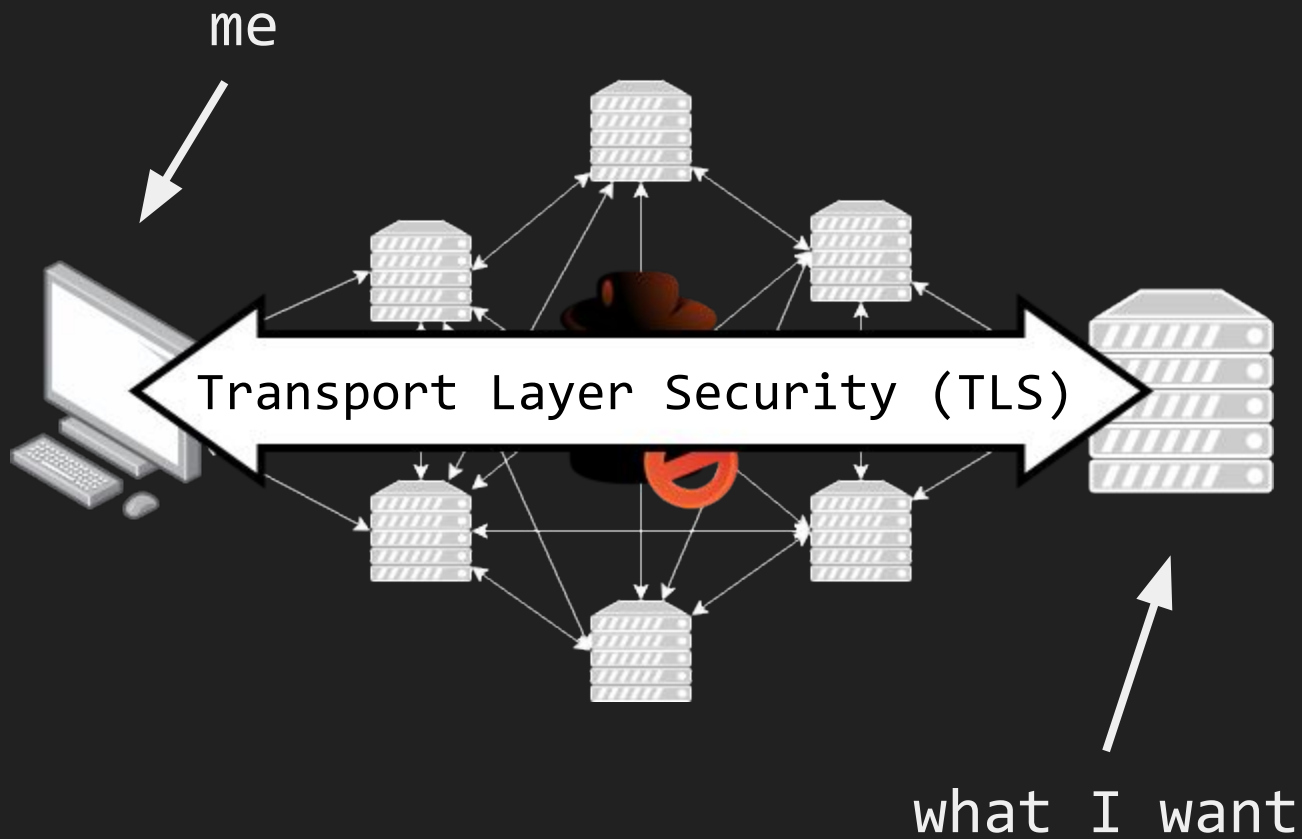


TLS

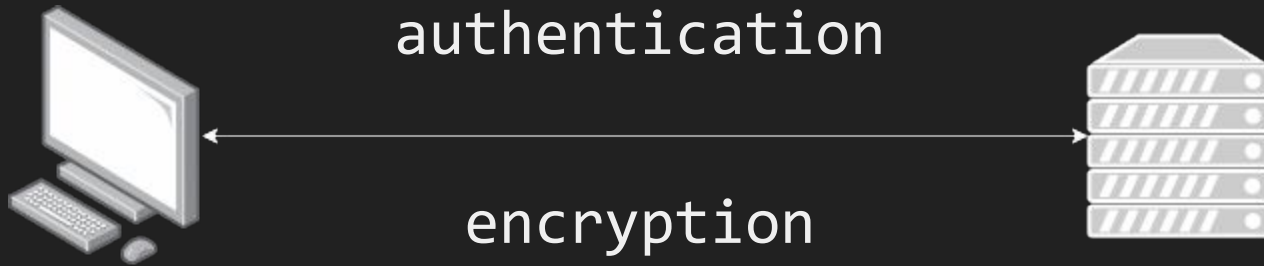




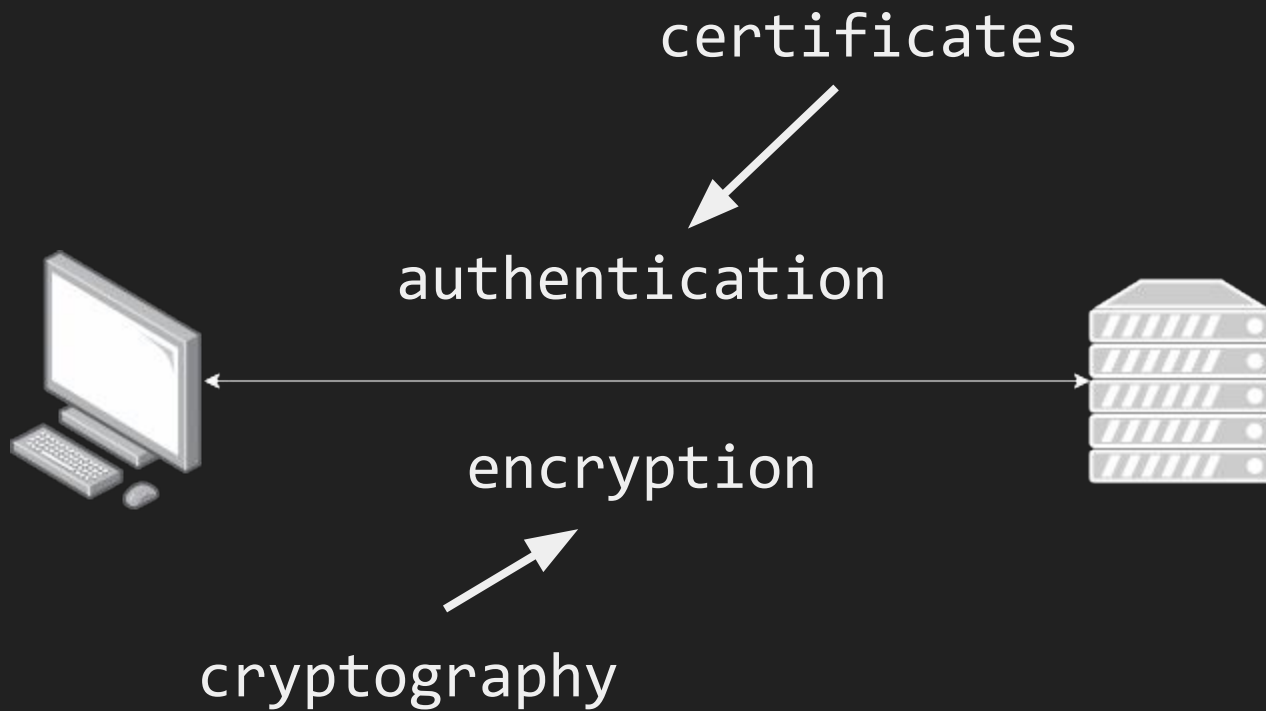




TLS:



TLS:



certificates

certificate



signature





certificate:

- issued by
- issued to
- public key
- misc
- signature

fancy text file

certificate:

- issued by: "aapo"
- issued to: "aapo"
- public key: aapo.pub
- misc: "this is aapo"
- signature: aapo.sig

private key:

- aapo.key

certificate:

- issued by: "aapo"
- issued to: "aapo"
- public key: aapo.pub
- misc: "this is aapo"
- signature: aapo.sig

private key:

- aapo.key

mathematical
proof of
ownership of
the public key





aapo.pem



aapo.key



aapo.pem



“issued to
aapo”
“issued by
aapo”



aapo.key

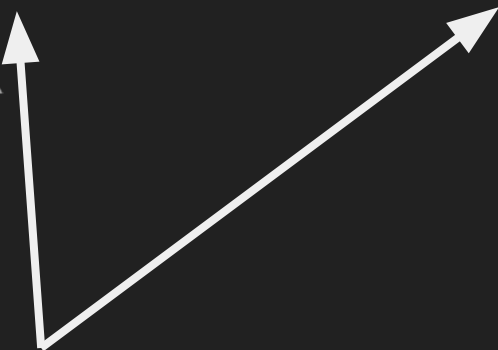
server.pem



“issued to
server”
“issued by
aapo”



server.key



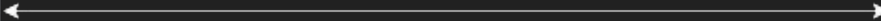
aapo.pem



server.pem



“issued
by
aapo”



server.key

aapo.pem



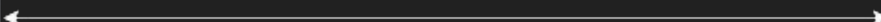
can check the
authenticity of server.pem



server.pem



“issued
by
aapo”



server.key

used as proof that the server
is allowed use server.pem



certificate validation

certificate validation problems in TLS

- multiple problems with TLS libraries and implementations over the years

certificate validation problems in TLS

- multiple problems with TLS libraries and implementations over the years
 - 2009-2012 sslsniff, sslstrip and more by Moxie Marlinspike @ Black Hat and DEF CON

certificate validation problems in TLS

- multiple problems with TLS libraries and implementations over the years
 - 2009-2012 sslsniff, sslstrip and more by Moxie Marlinspike @ Black Hat and DEF CON
- currently libraries work quite well

certificate validation problems in TLS

- multiple problems with TLS libraries and implementations over the years
 - 2009-2012 sslsniff, sslstrip and more by Moxie Marlinspike @ Black Hat and DEF CON
- currently libraries work quite well
- implementation is still hard

certificate validation problems in TLS

- multiple problems with TLS libraries and implementations over the years
 - 2009-2012 sslsniff, sslstrip and more by Moxie Marlinspike @ Black Hat and DEF CON
- currently libraries work quite well
- implementation is still hard
 - 2012 "The most dangerous code in the world: validating SSL certificates in non-browser software." Georgiev, Martin, et al.

aapo.pem



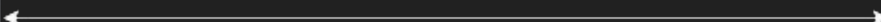
can check the
authenticity of server.pem



server.pem



“issued
by
aapo”



server.key

used as proof that the server
is allowed use server.pem



app_pos



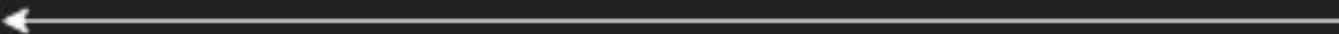
can check the
authenticity of server_pos



apo.pem



can check the
authenticity of server.pem



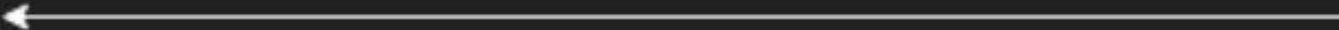
apo.pem



can check the
authenticity of server.pem



wtf means “can”?



certificate validation in TLS

- optional

certificate validation in TLS

- optional
 - “to ensure that you really are connected to the correct server you have to validate the server certificate. if you want to”
 - usually enabled by default

certificate validation in TLS

- optional
 - “to ensure that you really are connected to the correct server you have to validate the server certificate. if you want to”
 - usually enabled by default
 - can be turned off

certificate validation in TLS

- optional
 - “to ensure that you really are connected to the correct server you have to validate the server certificate. if you want to”
 - usually enabled by default
 - can be turned off = will be turned off

certificate validation in TLS

- optional
 - “to ensure that you really are connected to the correct server you have to validate the server certificate. if you want to”
 - usually enabled by default
 - can be turned off = will be turned off
- hard to configure
 - many libraries have insecure defaults

(practical) certificate validation in TLS

- (is certificate validation enabled?)
- has the certificate been signed?
- who has issued the certificate?
- who is the certificate issued to?

has the certificate been signed?

- has the private key of the “issuer” calculated a signature of the certificate contents
- usually checked by the TLS library

who has issued the certificate?

- A global certificate authority (CA)
 - GoDaddy
 - Let's encrypt
- 'self signed' (certificate has issued itself)
- is the issuer allowed to issue certificates?
- operating system CA list used by default

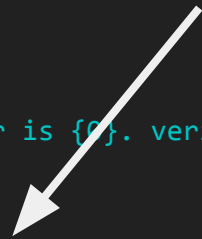
who is the certificate issued to?

- “aapo”
- “www.aapo.tld”
- hostname validation is not enabled by default on OpenSSL and WolfSSL

custom certificate validators

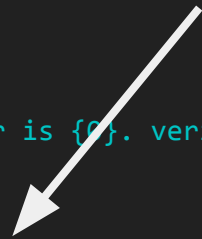
```
22. public static bool CertificateValidation(  
23.     object sender,  
24.     X509Certificate certificate,  
25.     X509Chain chain,  
26.     SslPolicyErrors sslPolicyErrors,  
27.     string[] trustedThumbprints)  
28. {  
29.     if (sslPolicyErrors == SslPolicyErrors.None) return true;  
30.  
31.     Log.TraceInformation("cert validation is failed, the error is {0}. verifying whether the server cert is  
trusted.", (object) sslPolicyErrors);  
32.     if (chain.ChainElements != null)  
33.     {  
34.         X509ChainElement chainElement = chain.ChainElements[chain.ChainElements.Count - 1];  
35.         if (chainElement != null && chainElement.Certificate != null)  
36.         {  
37.             X509Certificate2 certificate1 = chainElement.Certificate;  
38.             CertificateHelper.RootCert = certificate1;  
39.             if (CertificateHelper.thumbprints.Contains(certificate1.Thumbprint.ToUpper()))  
40.             {  
41.                 Log.TraceInformation("Root cert validation pass.");  
42.                 return true;  
43.             }  
44.             Log.TraceError("Thumbprint is not matched, the remote root cert thumbprint is {0}", (object)  
certificate1.Thumbprint);  
45.         }  
46.     }  
47.     return true;  
48. }
```

does not check who the
certificate is issued to



```
22. public static bool CertificateValidation(  
23.     object sender,  
24.     X509Certificate certificate,  
25.     X509Chain chain,  
26.     SslPolicyErrors sslPolicyErrors,  
27.     string[] trustedThumbprints)  
28. {  
29.     if (sslPolicyErrors == SslPolicyErrors.None) return true;  
30.  
31.     Log.TraceInformation("cert validation is failed, the error is {0}. verifying whether the server cert is  
32.     trusted.", (object) sslPolicyErrors);  
33.     if (chain.ChainElements != null)  
34.     {  
35.         X509ChainElement chainElement = chain.ChainElements[chain.ChainElements.Count - 1];  
36.         if (chainElement != null && chainElement.Certificate != null)  
37.         {  
38.             X509Certificate2 certificate1 = chainElement.Certificate;  
39.             CertificateHelper.RootCert = certificate1;  
40.             if (CertificateHelper.thumbprints.Contains(certificate1.Thumbprint.ToUpper()))  
41.             {  
42.                 Log.TraceInformation("Root cert validation pass.");  
43.                 return true;  
44.             }  
45.             Log.TraceError("Thumbprint is not matched, the remote root cert thumbprint is {0}", (object)  
46.                 certificate1.Thumbprint);  
47.         }  
48.     }  
49.     return true;  
50. }
```


does not check who the
certificate is issued to



```
22. public static bool CertificateValidation(  
23.     object sender,  
24.     X509Certificate certificate,  
25.     X509Chain chain,  
26.     SslPolicyErrors sslPolicyErrors,  
27.     string[] trustedThumbprints)  
28. {  
29.     if (sslPolicyErrors == SslPolicyErrors.None) return true;  
30.  
31.     Log.TraceInformation("cert validation is failed, the error is {0}. verifying whether the server cert is  
32.     trusted.", (object) sslPolicyErrors);  
33.     if (chain.ChainElements != null)  
34.     {  
35.         X509ChainElement chainElement = chain.ChainElements[chain.ChainElements.Count - 1];  
36.         if (chainElement != null && chainElement.Certificate != null)  
37.         {  
38.             X509Certificate2 certificate1 = chainElement.Certificate;  
39.             CertificateHelper.RootCert = certificate1;  
40.             if (CertificateHelper.thumbprints.Contains(certificate1.Thumbprint.ToUpper()))  
41.             {  
42.                 Log.TraceInformation("Root cert validation pass.");  
43.                 return true;  
44.             }  
45.             Log.TraceError("Thumbprint is not matched, the remote root cert thumbprint is {0}", (object)  
46.                 certificate1.Thumbprint);  
47.         }  
48.     }  
49.     return true;  
50. }
```



does not check
the signature

does not check who the
certificate is issued to

```
22. public static bool CertificateValidation(  
23.     object sender,  
24.     X509Certificate certificate,  
25.     X509Chain chain,  
26.     SslPolicyErrors sslPolicyErrors,  
27.     string[] trustedThumbprints)  
28. {  
29.     if (sslPolicyErrors == SslPolicyErrors.None) return true;  
30.  
31.     Log.TraceInformation("cert validation is failed, the error is {0}. verifying whether the server cert is  
32.     trusted.", (object) sslPolicyErrors);  
33.     if (chain.ChainElements != null)  
34.     {  
35.         X509ChainElement chainElement = chain.ChainElements[chain.ChainElements.Count - 1];  
36.         if (chainElement != null && chainElement.Certificate != null)  
37.         {  
38.             X509Certificate2 certificate1 = chainElement.Certificate;  
39.             CertificateHelper.RootCert = certificate1;  
40.             if (CertificateHelper.thumbprints.Contains(certificate1.Thumbprint.ToUpper()))  
41.             {  
42.                 Log.TraceInformation("Root cert validation pass.");  
43.                 return true;  
44.             }  
45.             Log.TraceError("Thumbprint is not matched, the remote root cert thumbprint is {0}", (object)  
46.                 certificate1.Thumbprint);  
47.         }  
48.     }  
49.     return true;  
50. }
```

always accepts
the certificate

does not check
the signature

```
5. // Assembly location: C:\Program Files (x86)\Microsoft Intune Management
Extension\Microsoft.Management.Services.IntuneWindowsAgent.AgentCommon.dll
6.
7. using System.Collections.Generic;
8. using System.Net.Security;
9. using System.Security.Cryptography.X509Certificates;
10.
11. namespace
Microsoft.Management.Services.IntuneWindowsAgent.AgentCommon.CommonHelper
12. {
13.     public class CertificateHelper
14.     {
15.         private static List<string> thumbprints = new List<string>()
16.         {
17.             "D4DE20D05E66FC53FE1A50882C78DB2852CAE474"
18.         };
19.
20.         public static X509Certificate2 RootCert { get; set; }
21.
22.         public static bool CertificateValidation(
```

CVE-2021-31980

Microsoft Intune Management
Extension Remote Code Execution
Vulnerability

Acknowledgements:
Aapo Oksman of Nixu Cybersecurity

attacks against certificate
validation

attacks against certificate validation in TLS

- who has issued the certificate?
- who is the certificate issued to?
- has the certificate been signed?

attacks against certificate validation in TLS

- who has issued the certificate?
 - let's self sign the certificate
- who is the certificate issued to?
- has the certificate been signed?

attacks against certificate validation in TLS

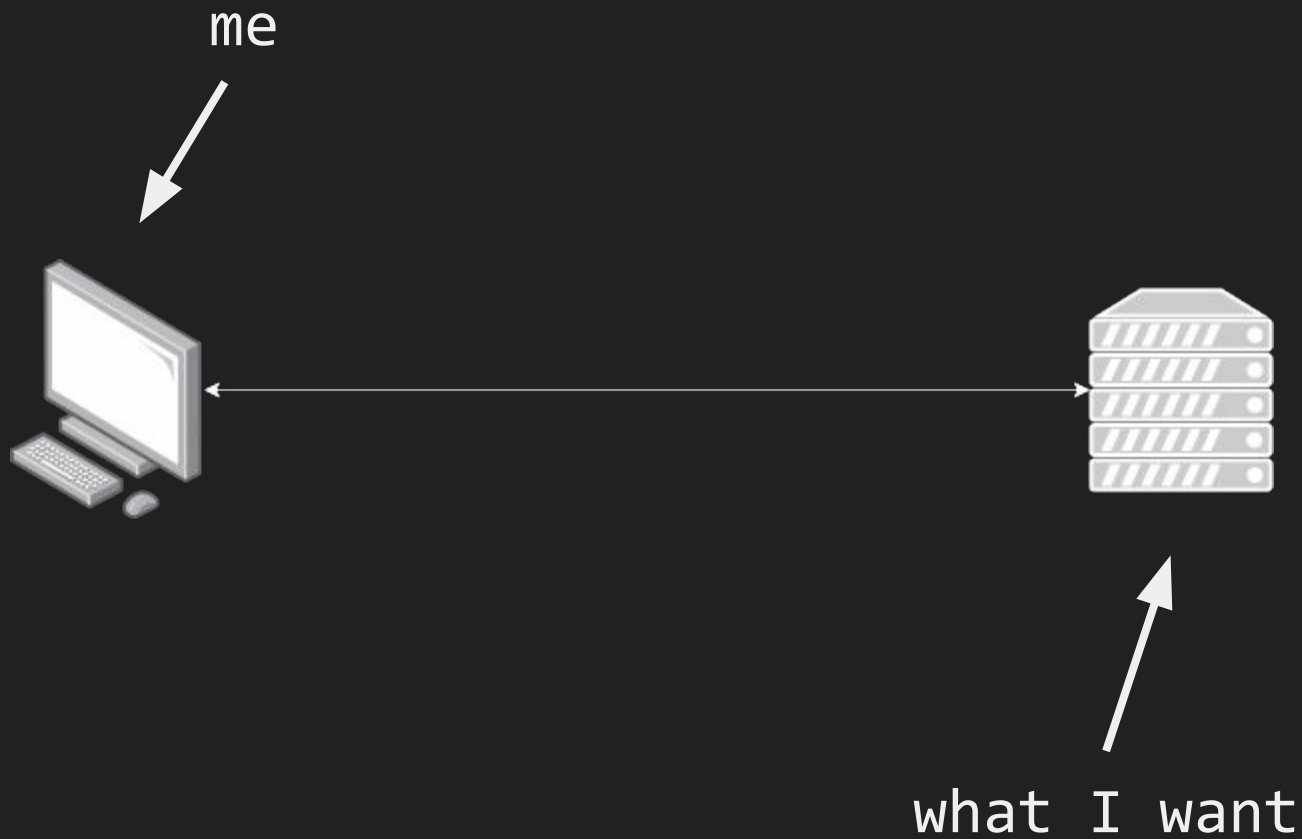
- who has issued the certificate?
 - let's self sign the certificate
 - let's sign the certificate with a random one
- who is the certificate issued to?
- has the certificate been signed?

attacks against certificate validation in TLS

- who has issued the certificate?
 - let's self sign the certificate
 - let's sign the certificate with a random one
- who is the certificate issued to?
 - let's get a valid certificate for our own hostname from a proper CA
- has the certificate been signed?

attacks against certificate validation in TLS

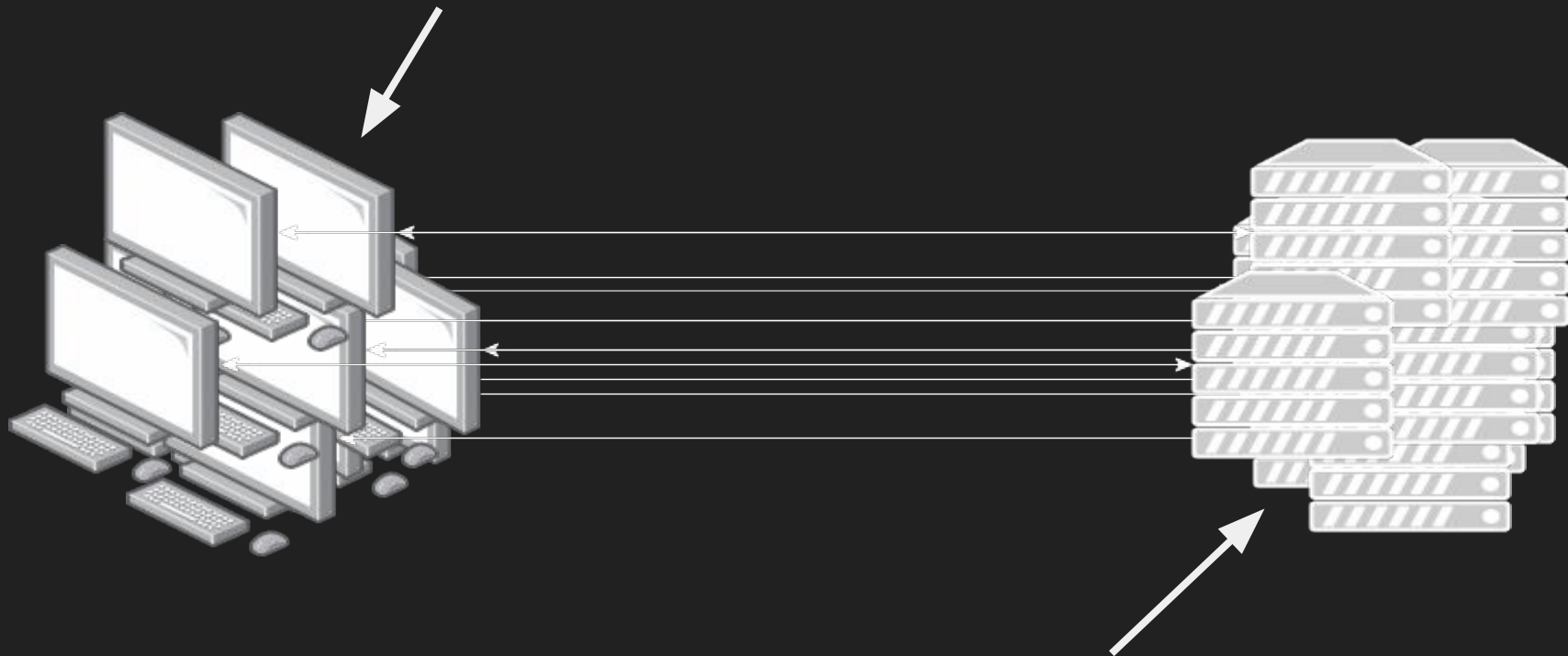
- who has issued the certificate?
 - let's self sign the certificate
 - let's sign the certificate with a random one
- who is the certificate issued to?
 - let's get a valid certificate for our own hostname from a proper CA
- has the certificate been signed?
 - let's change a valid certificate to have our public key -> the signature will be invalid







bunch of apps and libraries



what they happen to want

bunch of apps and libraries

so many certificate validators



what they happen to want

automation

automating certificate validation attacks

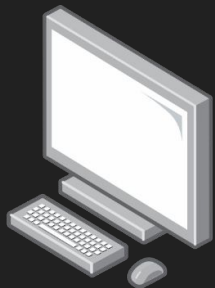
- each connection might have a different certificate validator

automating certificate validation attacks

- each connection might have a different certificate validator
- how to differentiate the connections?

automating certificate validation attacks

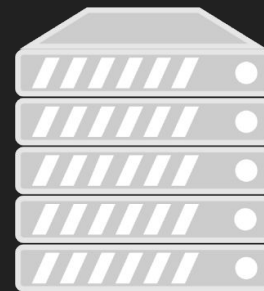
- each connection might have a different certificate validator
- how to differentiate the connections?
 - client ip
 - server ip
 - SNI hostname



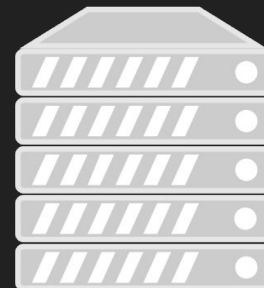
auth.exam.ple



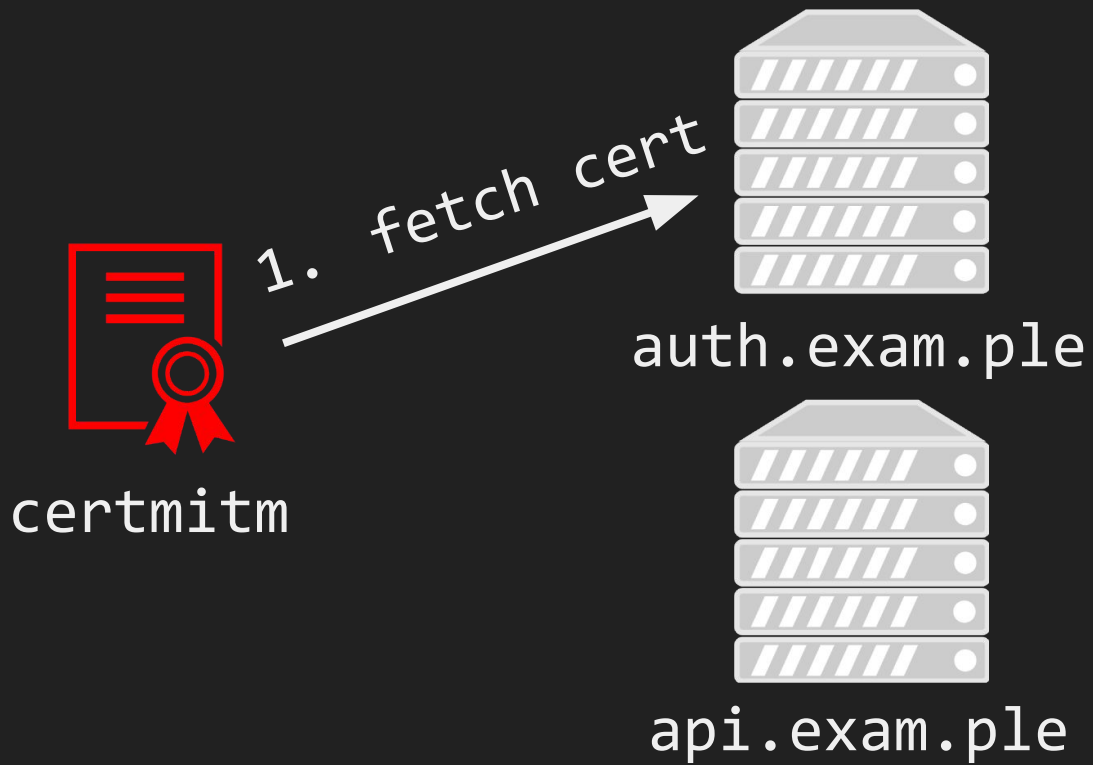
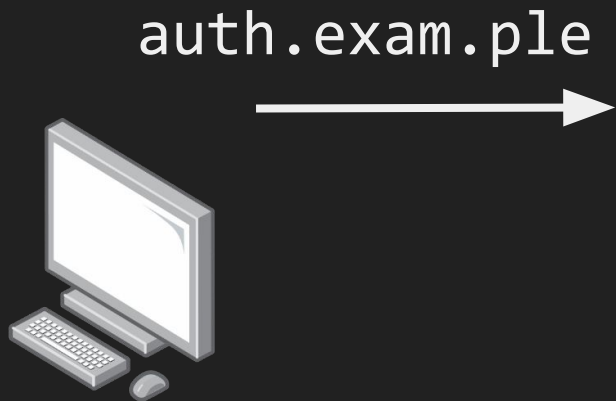
certmitm

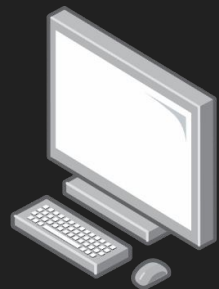


auth.exam.ple



api.exam.ple





auth.exam.ple

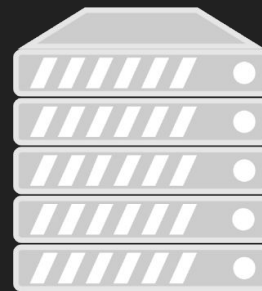


certmitm

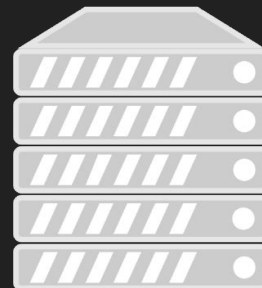
2. generate certs



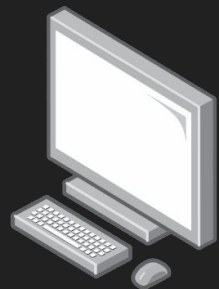
1. fetch cert



auth.exam.ple



api.exam.ple



auth.exam.ple

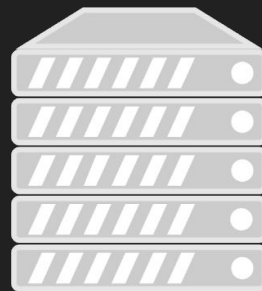


certmitm

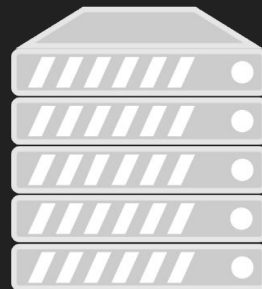
3. test certs

2. generate certs

1. fetch cert

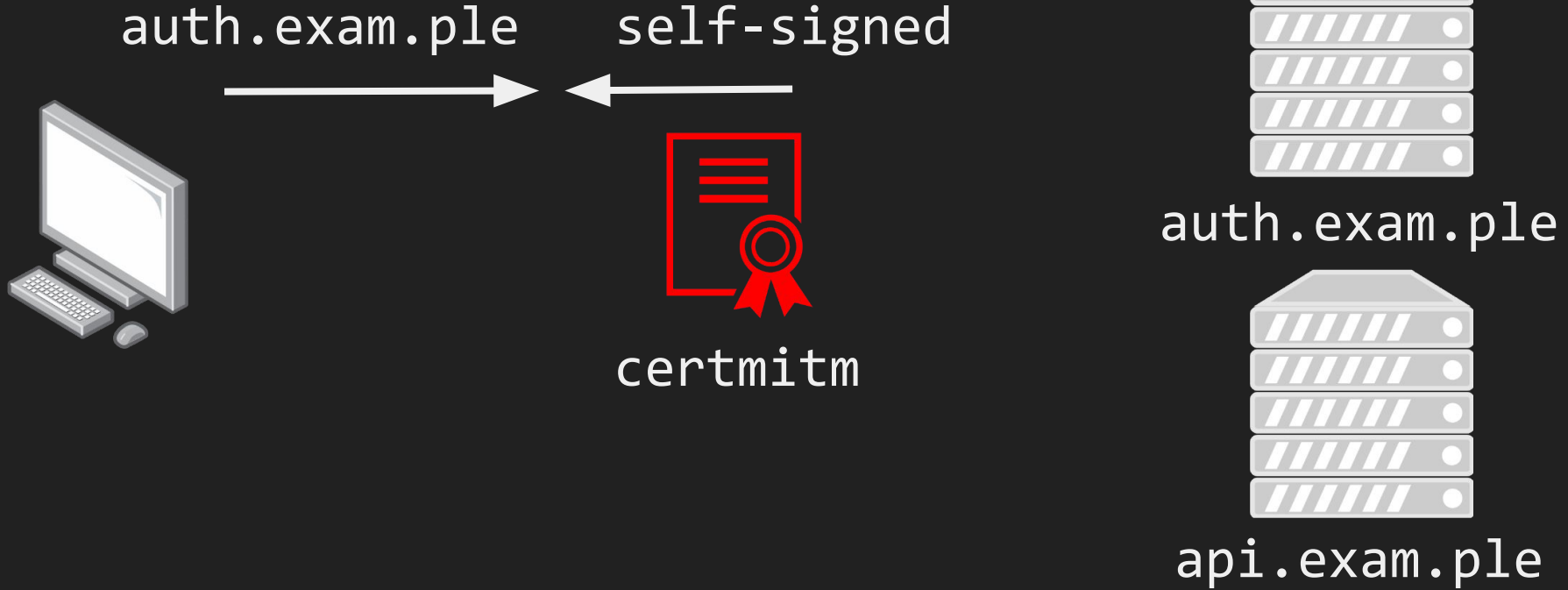


auth.exam.ple

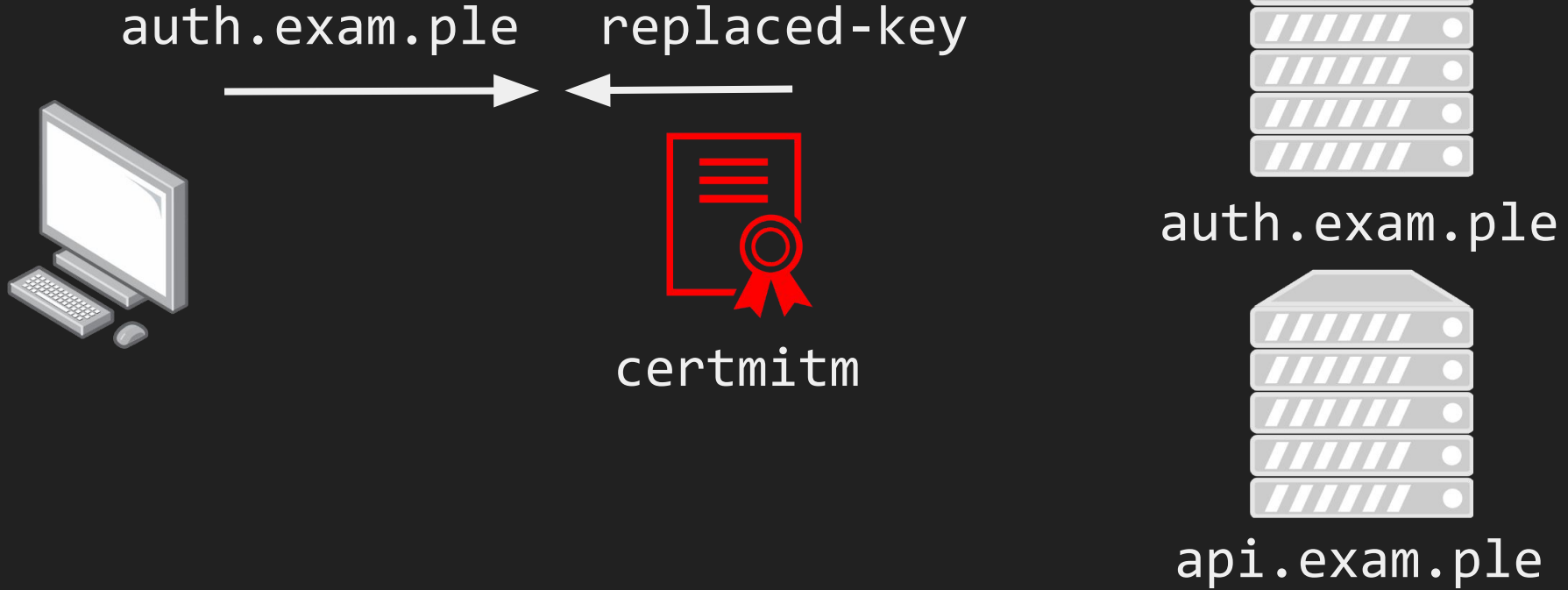


api.exam.ple

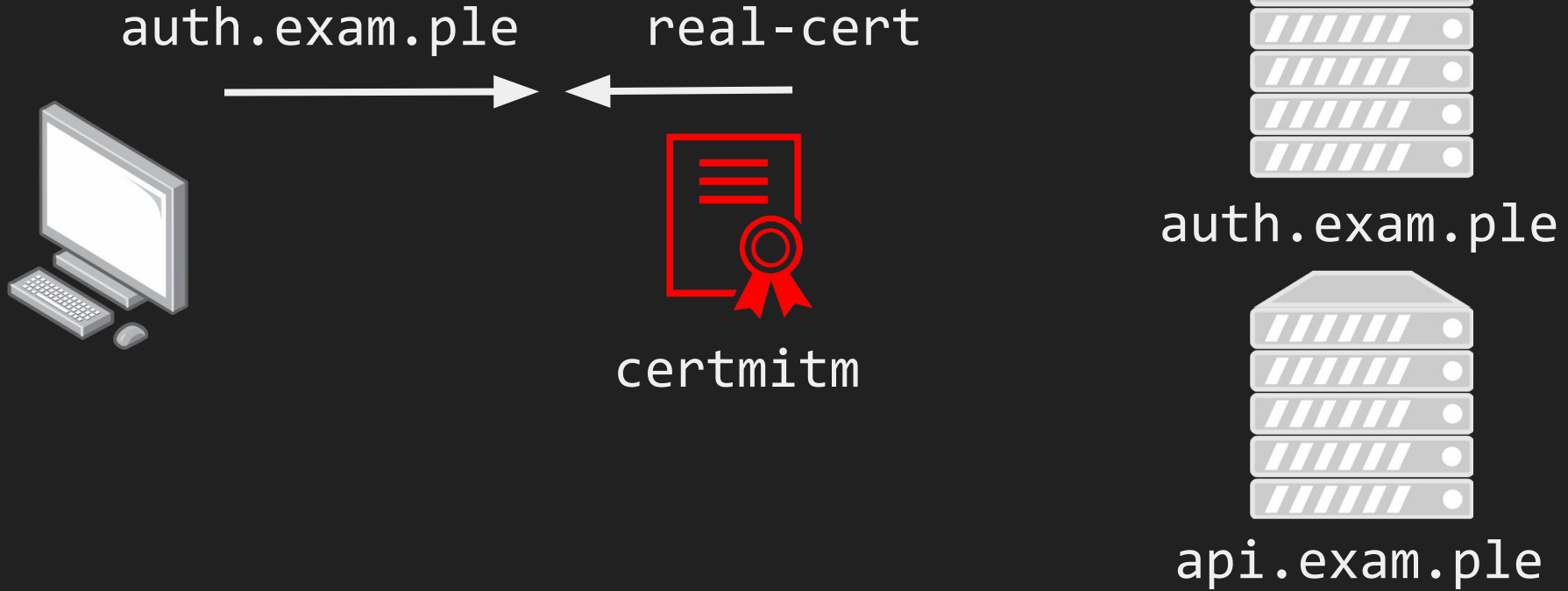
3. test certs



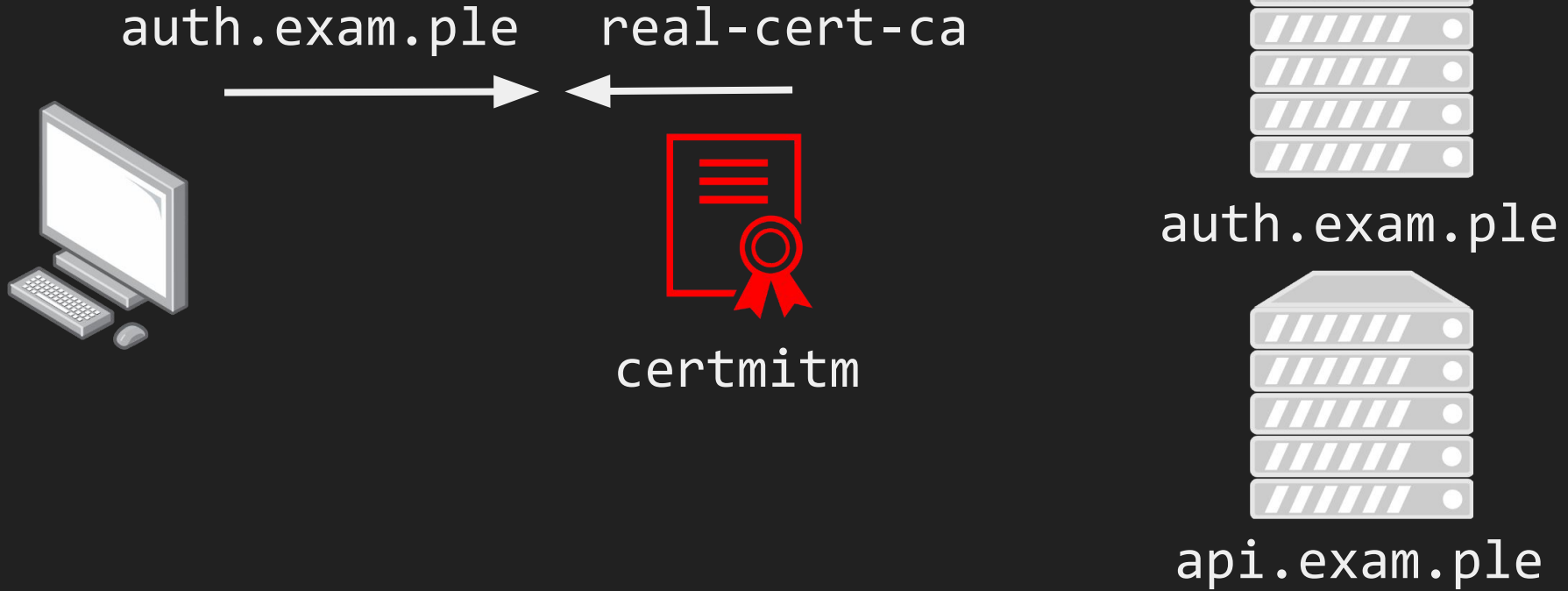
3. test certs



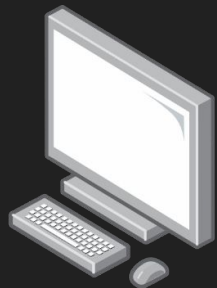
3. test certs



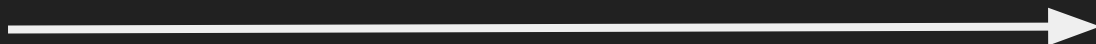
3. test certs



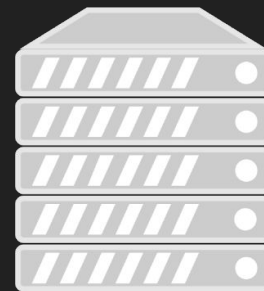
4. let it go



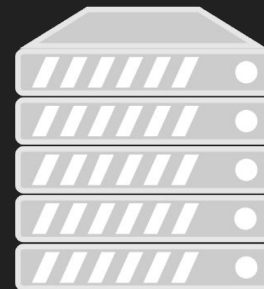
auth.exam.ple



certmitm

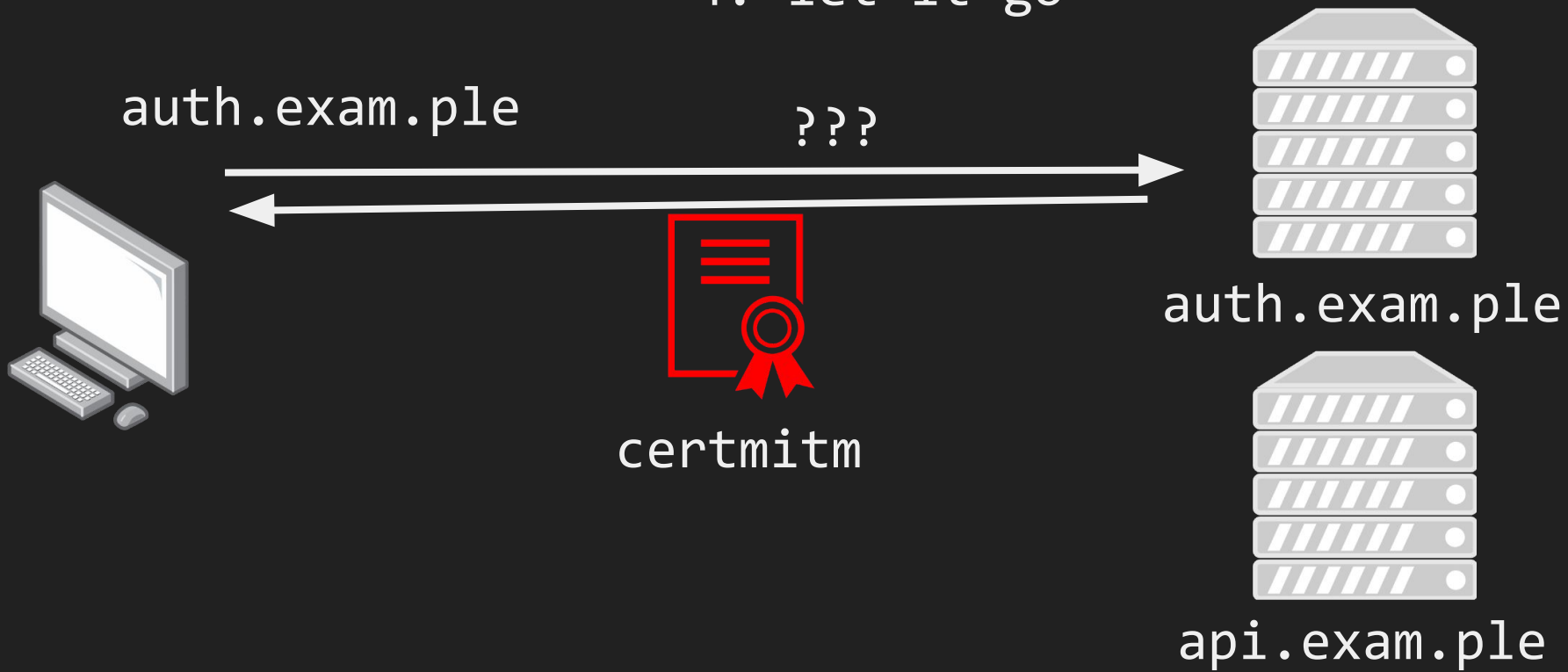


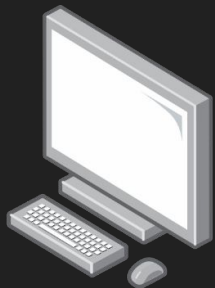
auth.exam.ple



api.exam.ple

4. let it go

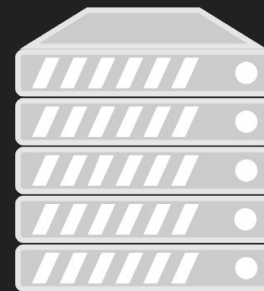




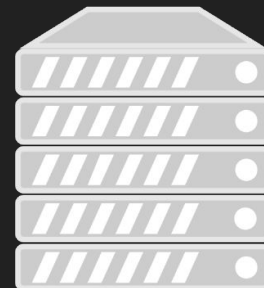
api.exam.ple



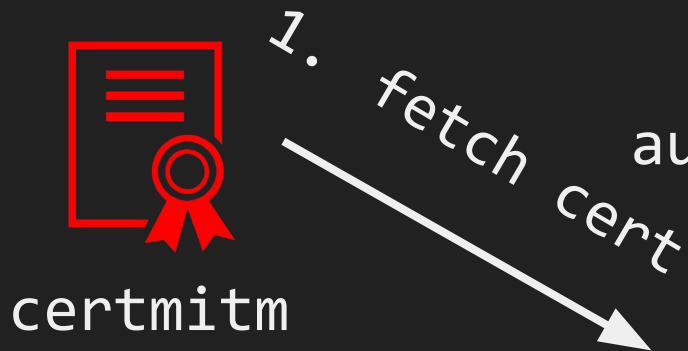
certmitm

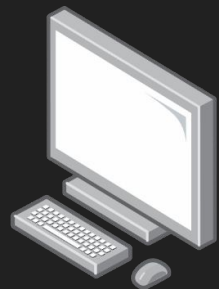


auth.exam.ple

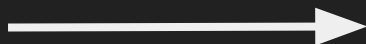


api.exam.ple





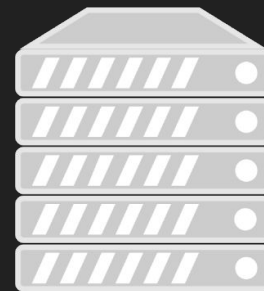
api.exam.ple



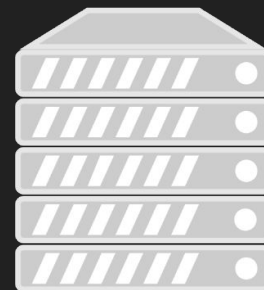
certmitm

2. generate certs

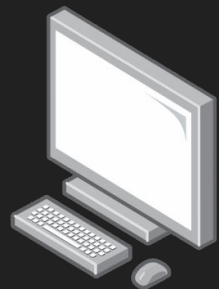
1. fetch cert



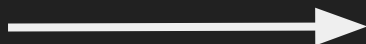
auth.exam.ple



api.exam.ple



api.exam.ple

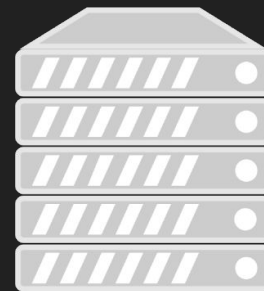


certmitm

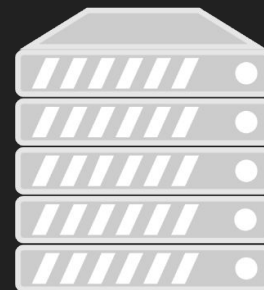
3. test certs

2. generate certs

1. fetch cert



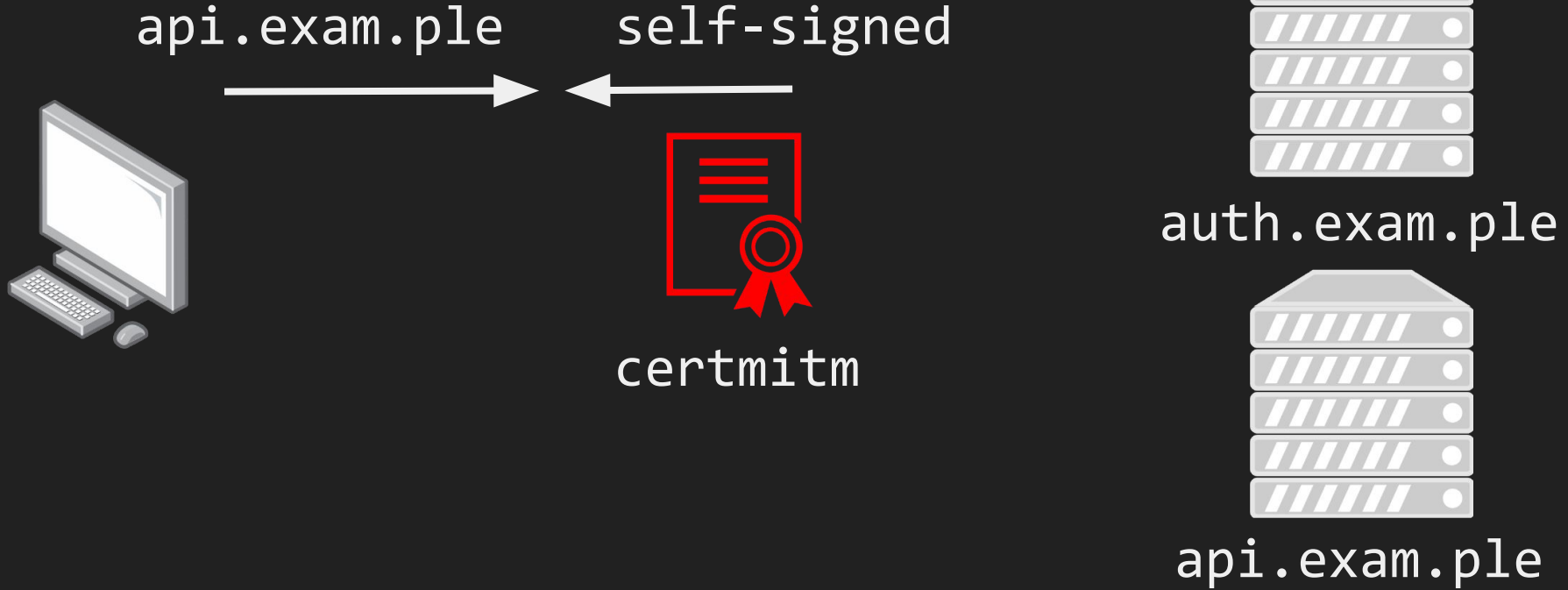
auth.exam.ple



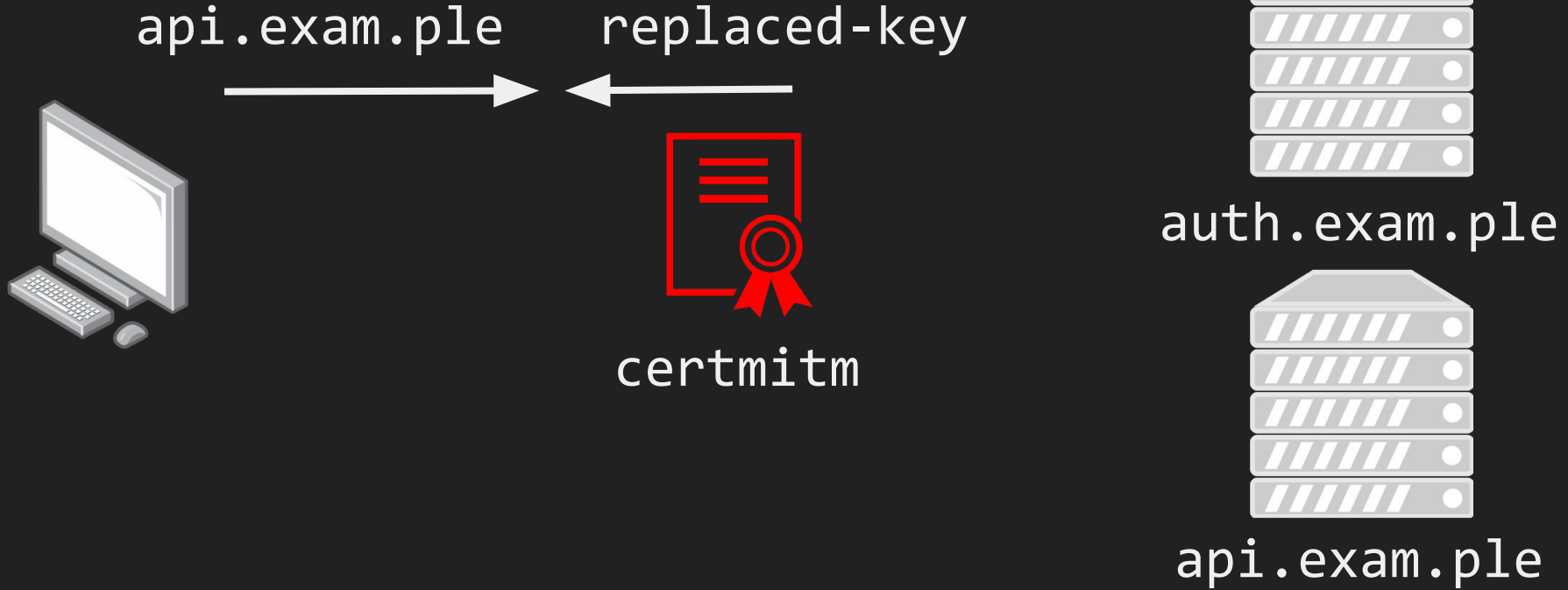
api.exam.ple



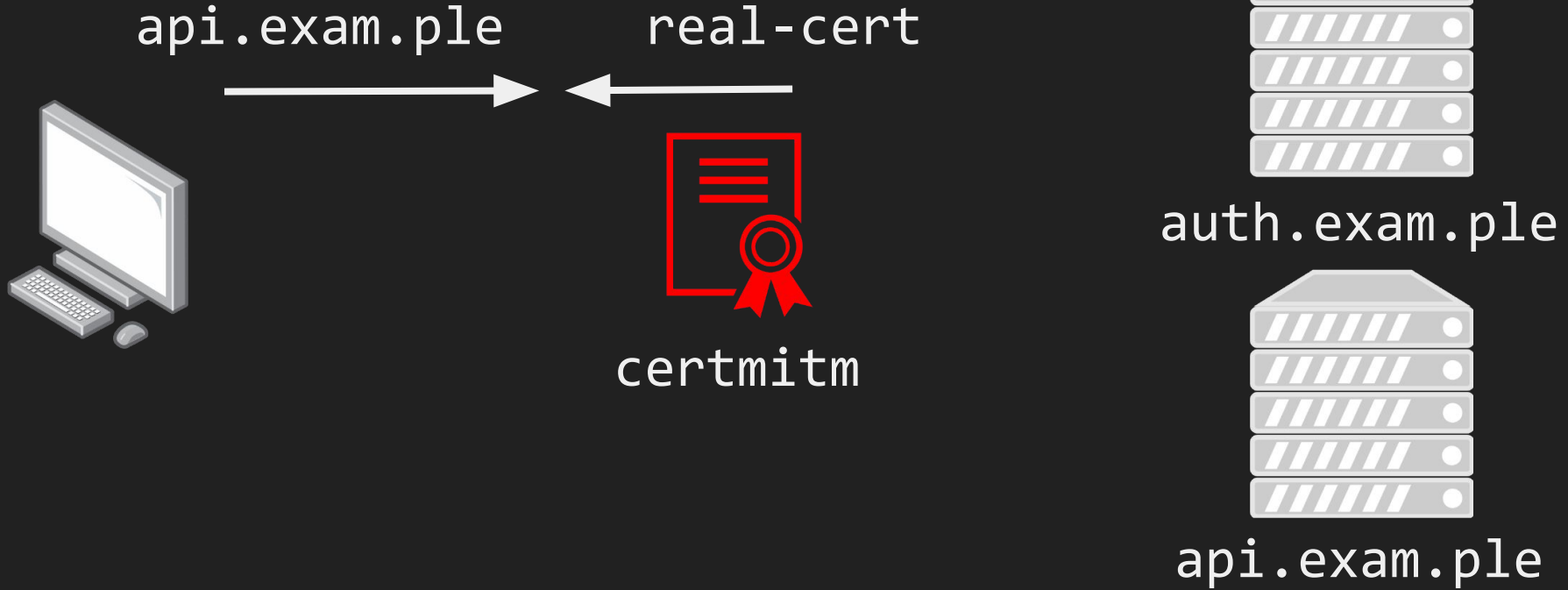
3. test certs



3. test certs



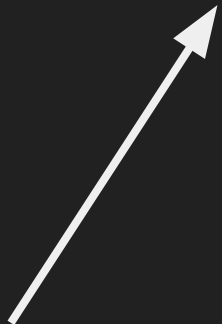
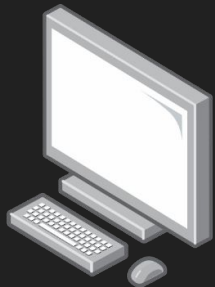
3. test certs



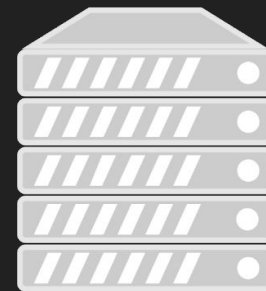
(3½. profit)

api.exam.ple

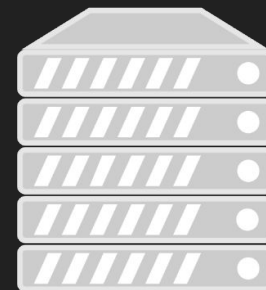
real-cert



certmitm



auth.exam.ple



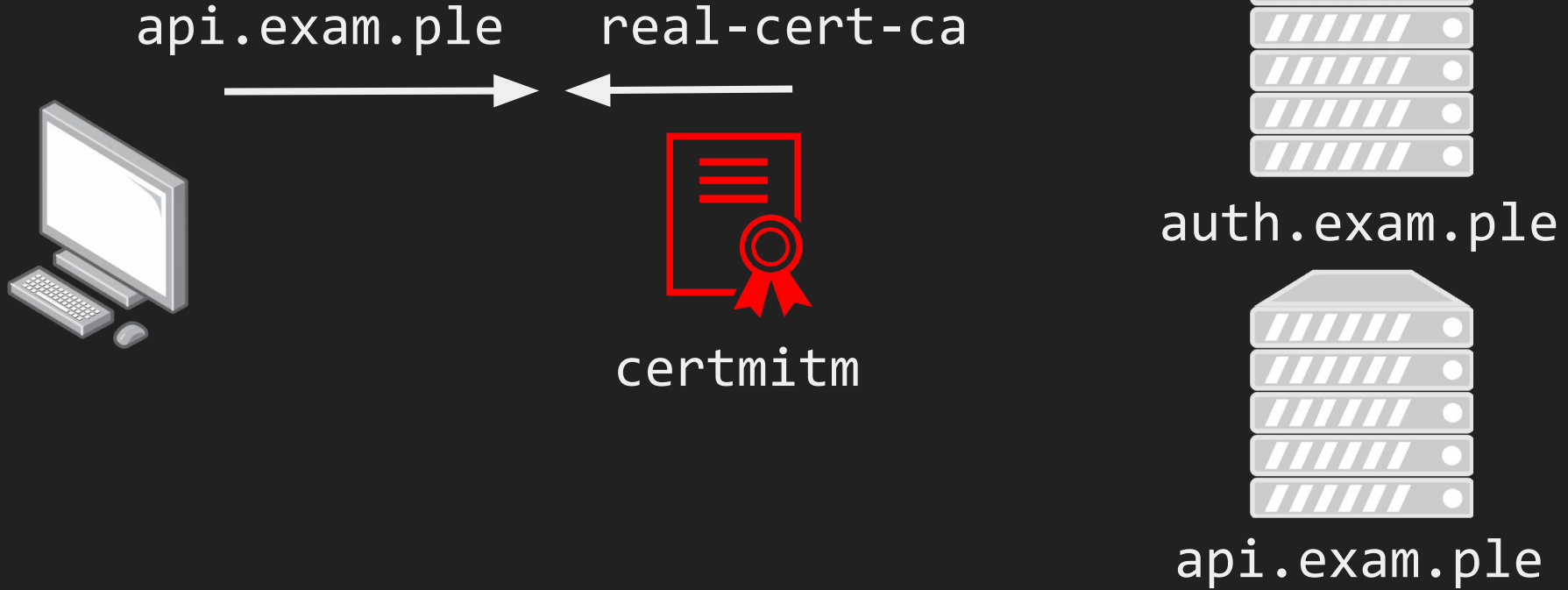
api.exam.ple

POST /payment

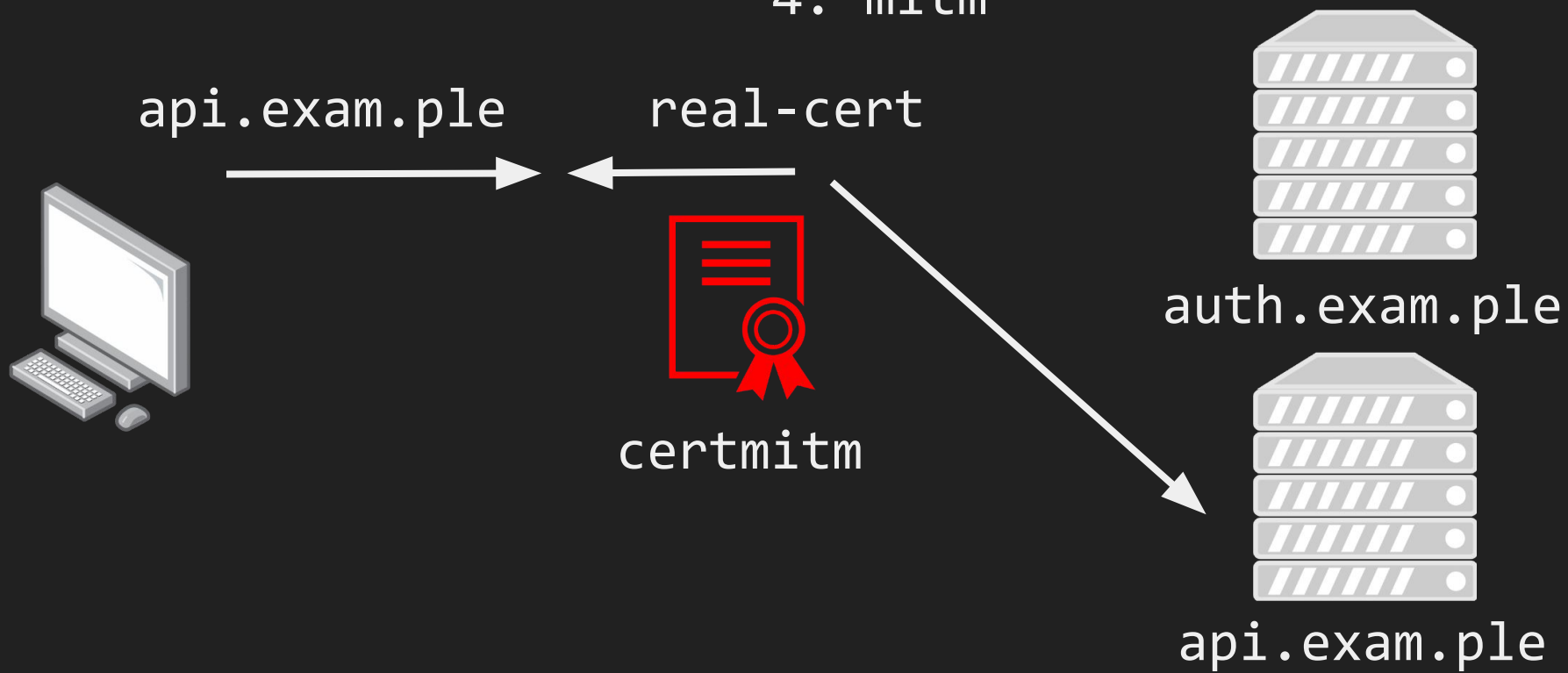
Authorization: Bearer eyylmao

{“amount”:”\$1000”,”to”:”bob”}

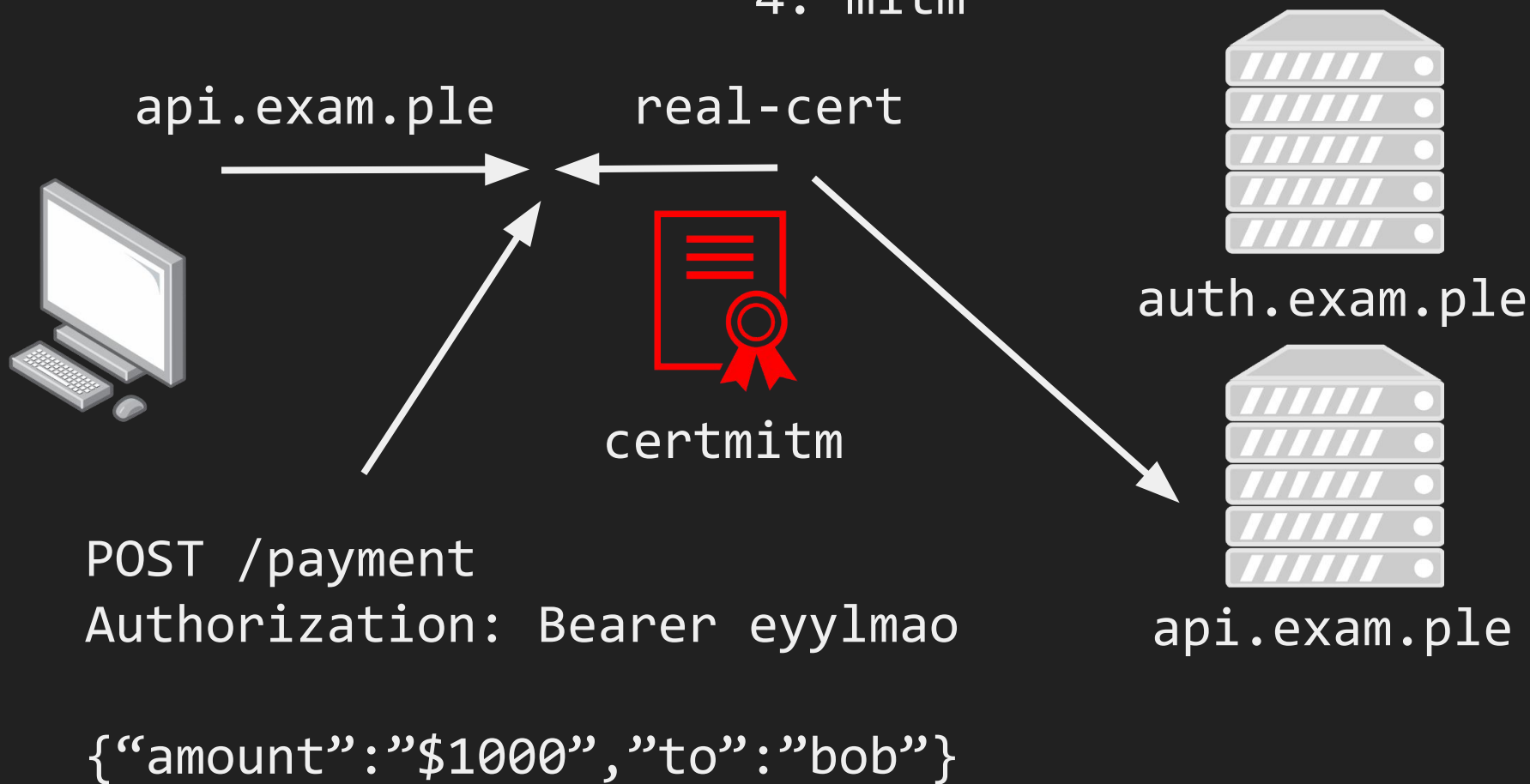
3. test certs



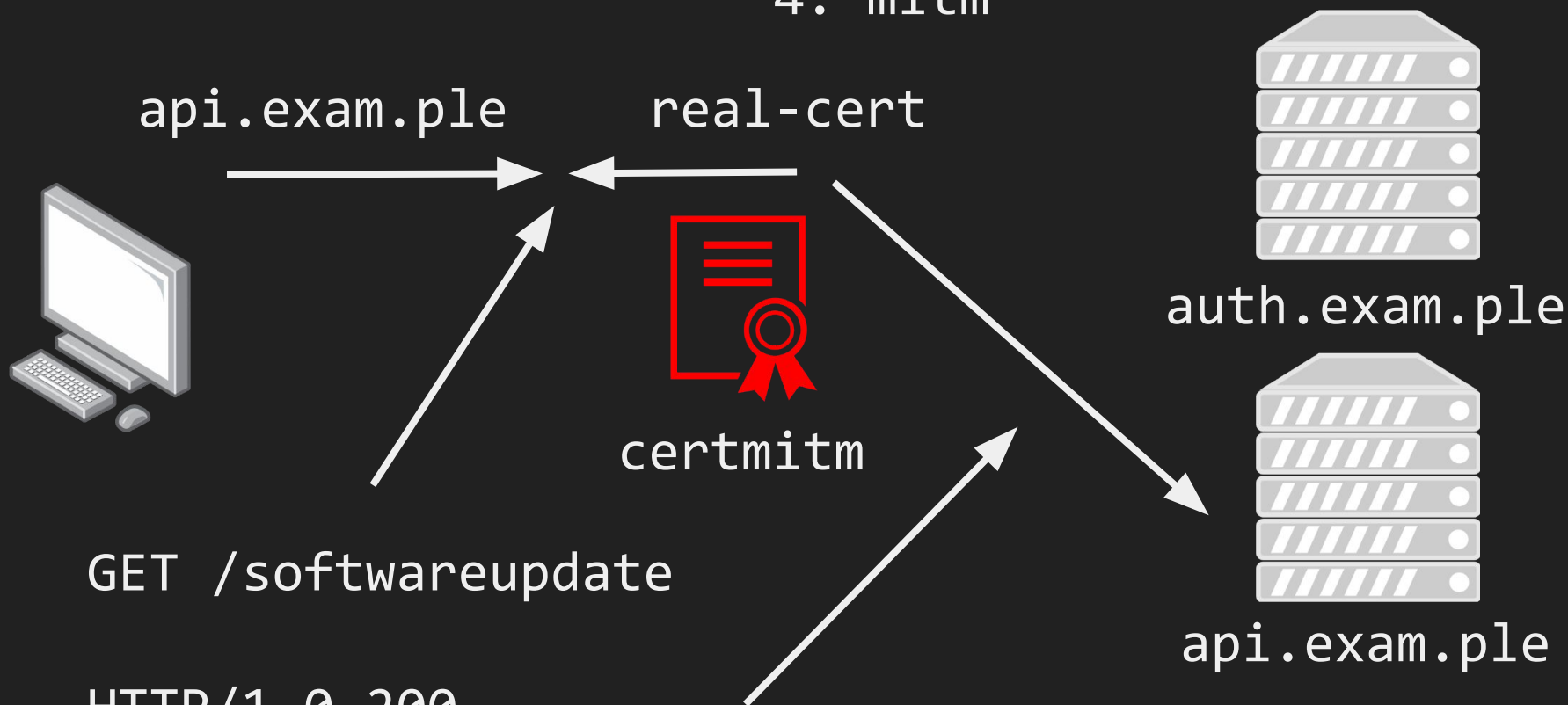
4. mitm



4. mitm



4. mitm



GET /softwareupdate

HTTP/1.0 200

Content-Type: application/octet-stream

certmitm demo

CVE:s

- CVE-2020-16971: Microsoft Azure SDK for Java
- CVE-2020-17002: Microsoft Azure SDK for C
- CVE-2021-1837: Apple iOS 14.4 and prior
- CVE-2021-31980: Microsoft Intune Management Extension
- CVE-2021-32581: Acronis True Image 2021
- CVE-2022-26913: Microsoft Windows Authentication
- CVE-2023-3615: Mattermost iOS app
- CVE-2023-28226: Microsoft Windows Enroll Engine
- CVE-2023-XXXXX:
- CVE-2023-YYYYY: More arriving shortly

and more

- ~80 vulnerable applications found on Android, iOS, Windows, Mac, game consoles, IoT etc.
- ~40 known 0-days currently
 - mostly on minor apps (couple big ones :)
 - some fixes rolling out in the coming weeks
- more vulnerabilities found every week

what can I do?

developers

- read the docs for your TLS library/framework
- no custom certificate validators
 - built-in validators support everything needed
 - (or there most likely is a reason your use case is not supported)
- check the hostname of the certificate
- unit tests
 - badssl.com has similar tests online and can be deployed on-prem
- run certmitm against your products

bug bounty hunters & pentesters

- run certmitm against everything
- find vulnerabilities
- profit?

bug bounty hunters & pentesters

- run certmitm against everything
 - yes
- find vulnerabilities
- profit?

bug bounty hunters & pentesters

- run certmitm against everything
 - yes
- find vulnerabilities
 - yes
- profit?

bug bounty hunters & pentesters

- run certmitm against everything
 - yes
- find vulnerabilities
 - yes
- profit?
 - yes and no

yes

“Your report qualifies for the Apple Security Bounty. Apple is pleased to award you with **\$10,000** for reporting this issue.”

“Remote Demonstrated unauthenticated and unauthorized access to private user data with little or no user interaction Attack Scenario for **\$50,000**. The man in the middle attack will qualify for the little user interaction portion of this attack scenario.”

“we have determined that your case is in-scope for a **\$10,000** bounty award under the Azure Bounty Program.”

no

“this issue does not qualify for a bounty or a CVE as the exploitation requires active Man in the middle”

no

“this issue does not qualify for a bounty or a CVE as the exploitation requires active Man in the middle”

“this does not pose an immediate threat that requires urgent attention since this requires that the victim's machine has been set up to query DNS from the attacker.”

no

“this issue does not qualify for a bounty or a CVE as the exploitation requires active Man in the middle”

“this does not pose an immediate threat that requires urgent attention since this requires that the victim's machine has been set up to query DNS from the attacker.”

“this is explicitly out of scope as outlined in the Policy Page: SSL/TLS Best Practices”

no

“this issue does not qualify for a bounty or a CVE as the exploitation requires active Man in the middle”

“this does not pose an immediate threat that requires urgent attention since this requires that the victim's machine has been set up to query DNS from the attacker.”

“this is explicitly out of scope as outlined in the Policy Page: SSL/TLS Best Practices“

“If this was an issue, it would be an issue with the public cloud provider, AWS and/or GCP, who are providing TLS termination services (its not our code layer). Negative points should be awarded for these types of submissions”

bug bounty ~~hunters~~ platforms

- mitm and TLS are out of scope on most programs

bug bounty ~~hunters~~ platforms

- mitm and TLS are out of scope on most programs
 - works for web application targets

bug bounty ~~hunters~~ platforms

- mitm and TLS are out of scope on most programs
 - works for web application targets
 - triagers refuse to accept certmitm reports even for client applications

bug bounty ~~hunters~~ platforms

- mitm and TLS are out of scope on most programs
 - works for web application targets
 - triagers refuse to accept certmitm reports even for client applications
 - many programs still accept the reports
 - (3/4 of the “no”s paid a bounty)

bug bounty ~~hunters~~ platforms

- mitm and TLS are out of scope on most programs
 - works for web application targets
 - triagers refuse to accept certmitm reports even for client applications
 - many programs still accept the reports
 - (3/4 of the “no”s paid a bounty)
- proposal: insecure certificate validation should be in scope for TLS clients when impact can be demonstrated

TLS library developers

2012 "The most dangerous code in the world: validating SSL certificates in non-browser software." Georgiev, Martin, et al.

"Future research directions include

- (1) development of better black-box testing and code analysis tools for discovering errors in SSL connection establishment logic
- (2) design of formal verification techniques and programming language support for automatically checking whether applications use SSL libraries correctly and not misinterpret the meaning of critical options and parameters
- (3) design of better APIs for SSL and other secure networking protocols."

aapo (oksman)

find certmitm @

<https://github.com/AapoOksman/certmitm>

find me @

- DEF CON
- nixu.com
- [linkedin.com/in/AapoOksman](https://www.linkedin.com/in/AapoOksman)
- aapo @ bug bounty

questions?